

Laravel Internship Viva Handbook

Week 1 — PHP & Laravel Fundamentals

Internship Duration: 4 Weeks

Week Covered: Week 1

Topics: Web Development, Client–Server Architecture, HTTP/HTTPS, MVC, XAMPP, Composer, VS Code, PHP Fundamentals, PHP OOP, Laravel Fundamentals, Artisan, Project Structure, Environment Configuration, Routing, Controllers, Blade Templates, and Passing Data.

Table of Contents

1. Week 1 Learning Objectives
2. Day 1 — Web Development Fundamentals
 - Web Development
 - Frontend and Backend
 - Client and Server
 - Client–Server Architecture
 - Request and Response
 - HTTP
 - HTTPS
 - HTTP Methods
 - Status Codes
 - MVC Architecture
 - XAMPP
 - Apache
 - PHP
 - MySQL
 - phpMyAdmin
 - Composer
 - VS Code
3. Day 2 — PHP Basics
 - PHP
 - PHP Syntax
 - Variables
 - Constants
 - Data Types
 - Operators
 - Conditional Statements
 - Loops
 - Arrays
 - Functions

- Scope
 - 4. Day 3 — PHP Object-Oriented Programming
 - OOP
 - Class
 - Object
 - Properties
 - Methods
 - Constructor
 - Encapsulation
 - Inheritance
 - Polymorphism
 - Abstraction
 - Traits
 - Namespaces
 - 5. Day 4 — Laravel Fundamentals
 - Laravel
 - Framework
 - Why Laravel
 - Installation
 - Composer and Laravel
 - Laravel Project Structure
 - Artisan
 - Important Artisan Commands
 - Environment Configuration
 - .env
 - config
 - 6. Day 5 — Routing, Controllers and Blade
 - Routing
 - Routes
 - HTTP Verbs
 - Route Parameters
 - Named Routes
 - Controllers
 - Controller Methods
 - Blade
 - Blade Directives
 - Passing Data
 - MVC Request Flow
 - 7. Week 1 Practical Project Flow
 - 8. Important Terms to Memorize
 - 9. Week 1 Viva Questions and Answers
 - 10. Rapid-Fire Viva Revision
-

1. Week 1 Learning Objectives

By the end of Week 1, you should understand:

- How a web application works.
- The difference between frontend and backend.
- How a browser communicates with a server.
- HTTP and HTTPS.
- HTTP request methods and status codes.
- MVC architecture.
- The purpose of XAMPP.
- The purpose of Composer.
- Basic PHP programming.
- Object-Oriented Programming in PHP.
- What Laravel is and why it is used.
- Laravel's basic project structure.
- Artisan commands.
- Environment configuration.
- Laravel routing.
- Controllers.
- Blade templates.
- How data moves from a controller to a view.

The most important goal is not memorizing code.

You should be able to explain:

What is this? Why is it used? How does it work? And where would I use it in a Laravel application?

DAY 1 — INTRODUCTION TO WEB DEVELOPMENT

1. What Is Web Development?

Web development is the process of designing, building, testing, and maintaining websites and web applications.

A web application normally consists of two major sides:

Frontend

The frontend is the part users see and interact with.

Common frontend technologies include:

- HTML
- CSS
- JavaScript
- Bootstrap

For example, on a Lost and Found application, the frontend includes:

- Navigation bar
- Search bar
- Item cards
- Forms
- Buttons
- Login page
- Dashboard

Backend

The backend handles the application's logic and data.

Common backend technologies include:

- PHP
- Laravel
- Python
- Java
- Node.js
- MySQL and other databases

For a Laravel application:

```
Browser
  ↓
Laravel
  ↓
PHP
  ↓
Database
```

The frontend provides the interface, while the backend processes requests and manages application data.

2. Website vs Web Application

A website primarily presents information.

Examples:

- Company website
- Portfolio
- Blog
- News website

A web application is more interactive and usually performs operations based on users and stored data.

Examples:

- Banking system
- Student management system
- E-commerce system
- Lost and Found system

A Laravel CRUD application is generally considered a web application because users interact with stored data.

3. Client and Server

This is one of the most important concepts in web development.

Client

The client is the software or device that requests a service from a server.

In normal web development, the browser acts as the client.

Examples:

- Google Chrome
- Microsoft Edge
- Firefox
- Safari

Server

A server receives requests, processes them, and sends responses.

In local Laravel development, your computer acts as the server.

For example:

```
Browser
  ↓
Request
  ↓
Laravel Application
  ↓
Response
  ↓
Browser
```

4. Client–Server Architecture

Client–server architecture divides responsibilities between the client and server.

Example

Suppose the user opens:

```
/items
```

The browser sends a request:

```
GET /items
```

Laravel receives it.

Laravel determines which route handles the request.

The controller retrieves the items from the database.

The controller sends the data to a Blade view.

The browser receives the generated HTML.

The complete flow is:

```
User
  ↓
```

```
Browser
↓
HTTP Request
↓
Web Server
↓
Laravel Route
↓
Controller
↓
Model / Database
↓
Controller
↓
Blade View
↓
HTTP Response
↓
Browser
↓
User
```

This flow is extremely important for understanding Laravel.

5. Request and Response

Request

A request is a message sent from the client to the server.

A request can contain:

- URL
- HTTP method
- headers
- query parameters
- form data
- JSON data
- authentication information

Example:

```
GET /items
```

Response

A response is the server's reply.

It may contain:

- HTML
- JSON
- files
- status code
- headers

Example:

200 OK

with an HTML page.

6. HTTP

HTTP stands for:

HyperText Transfer Protocol

HTTP is a communication protocol used between clients and web servers.

It defines how requests and responses are exchanged.

For example:

`GET /items`

means the client is requesting the items resource.

7. HTTPS

HTTPS stands for:

HyperText Transfer Protocol Secure

HTTPS is the secure version of HTTP.

It encrypts communication between the client and server.

This is especially important when transmitting sensitive information such as:

- Passwords
- Authentication tokens
- Personal information
- Payment information

HTTP

`http://example.com`

HTTPS

`https://example.com`

HTTPS protects information from being easily read or modified while travelling between the client and server.

8. HTTP Methods

HTTP methods indicate what the client wants to do.

GET

Used to retrieve data.

Example:

`GET /items`

Meaning:

Give me the items.

POST

Used to submit data or create a new resource.

Example:

`POST /items`

Meaning:

Create a new item.

PUT

Generally used to completely update or replace a resource.

Example:

```
PUT /items/5
```

Meaning:

Update item 5.

PATCH

Used for a partial update.

Example:

```
PATCH /items/5
```

Meaning:

Change only some fields of item 5.

DELETE

Used to delete a resource.

Example:

```
DELETE /items/5
```

Meaning:

Delete item 5.

9. HTTP Methods and CRUD

A useful relationship to memorize:

HTTP Method	CRUD Operation	Purpose
GET	Read	Retrieve data
POST	Create	Create data
PUT	Update	Replace/update resource
PATCH	Update	Partially update resource
DELETE	Delete	Delete resource

10. HTTP Status Codes

Status codes tell the client what happened to its request.

2xx — Successful

200 OK

Request was successful.

201 Created

A new resource was successfully created.

204 No Content

Request was successful but there is no response body.

4xx — Client Errors

400 Bad Request

The request was invalid.

401 Unauthorized

The client has not authenticated.

403 Forbidden

The client is authenticated but does not have permission.

404 Not Found

Requested resource does not exist.

422 Unprocessable Content

Usually used when submitted data fails validation.

5xx — Server Errors

500 Internal Server Error

Something went wrong on the server.

11. Authentication vs Authorization

These terms are frequently confused.

Authentication

Authentication answers:

Who are you?

Example:

A user logs in using:

Email
Password

The application verifies the user's identity.

Authorization

Authorization answers:

What are you allowed to do?

Example:

Admin → Can delete users
User → Cannot delete users

Easy way to remember:

Authentication = Who are you?

Authorization = What can you do?

12. MVC Architecture

MVC stands for:

Model – View – Controller

Laravel follows an MVC-based architecture.

Model

The Model represents application data and interacts with the database.

Examples:

User
Item
Category
Student
Course

In Laravel:

app/Models/Item.php

View

The View is responsible for presentation.

Laravel uses Blade templates.

Example:

```
resources/views/items/index.blade.php
```

A view contains HTML and dynamic Blade expressions.

Controller

The Controller handles application logic and coordinates between the request, models, and views.

Example:

```
app/Http/Controllers/ItemController.php
```

13. MVC Example

Suppose the user visits:

```
/items
```

The process can be:

```
Browser
  ↓
Route
  ↓
ItemController
  ↓
Item Model
  ↓
Database
  ↓
ItemController
  ↓
Blade View
  ↓
Browser
```

Model

Gets item data.

Controller

Decides what to do with that data.

View

Displays the data.

14. Why Use MVC?

MVC separates responsibilities.

Without separation, all database queries, HTML, and business logic could become mixed together.

MVC makes applications:

- Easier to understand
 - Easier to maintain
 - Easier to test
 - Easier to modify
 - Better organized
-

15. XAMPP

XAMPP is a local development environment commonly used for PHP development.

It commonly provides:

- Apache
 - MariaDB/MySQL
 - PHP
 - phpMyAdmin
-

16. Apache

Apache is a web server.

Its job is to receive HTTP requests and serve web applications.

In a traditional PHP/XAMPP setup:

```
Browser
↓
Apache
↓
PHP
↓
Application
```

17. PHP

PHP stands for:

PHP: Hypertext Preprocessor

PHP is a server-side scripting language.

It can:

- Process form data
- Connect to databases
- Perform calculations
- Manage sessions
- Generate HTML
- Implement application logic

Laravel is built using PHP.

18. MySQL / MariaDB

A database management system stores application data.

For example, a Lost and Found application could contain:

```
users
items
categories
claims
```


Data may look like:

Item ID: 1
Name: Wallet
Type: Lost
Location: Library

19. phpMyAdmin

phpMyAdmin is a web-based interface for managing MySQL/MariaDB databases.

It allows developers to:

- Create databases
 - Create tables
 - Insert records
 - Edit records
 - Delete records
 - Run SQL queries
-

20. Composer

Composer is the dependency manager for PHP.

A dependency is a package/library that an application requires.

Laravel uses Composer to install and manage its PHP dependencies.

Important file:

`composer.json`

This file describes project dependencies and other Composer configuration.

Another important file is:

`composer.lock`

It records the exact dependency versions installed for the project.

21. VS Code

Visual Studio Code is a source-code editor.

It can be used to work with:

- PHP
- Laravel
- Blade
- HTML
- CSS
- JavaScript
- SQL

It also provides extensions, syntax highlighting, debugging support, terminal integration, and source-control features.

DAY 2 — PHP BASICS

22. What Is PHP?

PHP is a server-side programming language commonly used for web development.

Example:

```
<?php
$name = "Saleha";
echo $name;
```

The PHP code runs on the server.

The browser receives the result rather than directly executing the PHP source code.

23. PHP Variables

Variables store data.

PHP variables begin with \$.

```
$name = "Saleha";  
$age = 20;
```

Here:

```
$name → variable  
"Saleha" → value
```

24. PHP Data Types

Common PHP data types include:

String

```
$name = "Saleha";
```

Integer

```
$age = 20;
```

Float

```
$price = 99.50;
```

Boolean

```
$isFound = true;
```

Array

```
$items = ["Wallet", "Phone", "Keys"];
```

Null

```
$value = null;
```

PHP also supports objects and other types.

25. Constants

A constant is a value that should not change during execution.

Example:

```
define("APP_NAME", "Foundly");
```

PHP also supports the `const` keyword.

Constants are useful for values that should remain fixed.

26. Operators

Operators perform operations on values.

Arithmetic Operators

+
-
*
/
%

Example:

```
$a = 10;  
$b = 3;  
  
$result = $a + $b;
```

Assignment Operator

```
$name = "Saleha";
```

= assigns a value.

Comparison Operators

==
===

```
!=  
!==  
>  
<  
>=  
<=
```

==

Checks equality after type conversion when applicable.

===

Checks equality of both value and type.

Example:

```
5 == "5"      // true  
5 === "5"     // false
```

27. Logical Operators

Common logical operators:

```
&&    AND  
||    OR  
!     NOT
```

Example:

```
if ($age >= 18 && $hasId) {  
    echo "Allowed";  
}
```

28. Conditional Statements

Conditional statements allow the program to make decisions.

if

```
if ($age >= 18) {  
    echo "Adult";  
}
```

if...else

```
if ($age >= 18) {  
    echo "Adult";  
} else {  
    echo "Minor";  
}
```

elseif

```
if ($marks >= 80) {  
    echo "A";  
} elseif ($marks >= 70) {  
    echo "B";  
} else {  
    echo "C";  
}
```

29. Loops

Loops repeat code.

for Loop

```
for ($i = 1; $i <= 5; $i++) {  
    echo $i;  
}
```

Useful when the number of repetitions is known.

while Loop

```
$i = 1;  
  
while ($i <= 5) {  
    echo $i;  
    $i++;  
}
```

Repeats while a condition is true.

foreach Loop

Very common when working with arrays and Laravel collections.

```
$items = ["Wallet", "Phone", "Keys"];

foreach ($items as $item) {
    echo $item;
}
```

30. Arrays

An array stores multiple values.

Indexed Array

```
$items = [
    "Wallet",
    "Phone",
    "Keys"
];
```

Indexes start at zero.

```
echo $items[0];
```

returns:

```
Wallet
```

Associative Array

Associative arrays use named keys.

```
$item = [
    "name" => "Wallet",
    "location" => "Library",
    "type" => "lost"
];
```

Access:

```
echo $item["name"];
```

31. Functions

A function is a reusable block of code.

```
function greet($name)
{
    return "Hello " . $name;
}
```

Use it:

```
echo greet("Saleha");
```

Advantages:

- Reusability
- Organization
- Less duplication
- Easier maintenance

32. Function Parameters and Return Values

```
function add($a, $b)
{
    return $a + $b;
}

$result = add(10, 20);
```

Here:

```
$a and $b → parameters
10 and 20 → arguments
$result → returned result
```

33. Variable Scope

Scope determines where a variable can be accessed.

Common scopes include:

- Local
- Global
- Static

Example:

```
function test()
```



```
{  
    $name = "Saleha";  
}
```

\$name is local to that function.

DAY 3 — PHP OBJECT-ORIENTED PROGRAMMING

34. What Is OOP?

OOP stands for:

Object-Oriented Programming

It is a programming approach based around objects and classes.

Laravel heavily relies on OOP.

For example:

```
User  
Item  
Category  
Controller  
Request  
Model
```

are represented through PHP classes.

35. Class

A class is a blueprint for creating objects.

```
class Student  
{  
    public $name;  
  
    public function introduce()  
    {  
        return "Hello";  
    }  
}
```

```
}  
}
```

The class describes what a Student object can contain and do.

36. Object

An object is an instance of a class.

```
$student = new Student();
```

Now `$student` is an object created from the `Student` class.

Think:

```
Class   = Blueprint  
Object = Actual instance
```

37. Properties

Properties are variables belonging to a class.

```
class Student  
{  
    public $name;  
    public $age;  
}
```

`$name` and `$age` are properties.

38. Methods

Methods are functions defined inside classes.

```
class Student  
{  
    public function study()  
    {  
        return "Student is studying";  
    }  
}
```

`study()` is a method.

39. Constructor

A constructor is a special method that runs automatically when an object is created.

```
class Student
{
    public $name;

    public function __construct($name)
    {
        $this->name = $name;
    }
}
```

Create the object:

```
$student = new Student("Saleha");
```

The constructor receives "Saleha" automatically.

40. `$this`

`$this` refers to the current object.

Example:

```
$this->name
```

means:

the `name` property belonging to this object.

41. Access Modifiers

PHP provides access modifiers to control visibility.

public

Accessible from anywhere where the object is accessible.

```
public $name;
```

private

Accessible only inside the class.

```
private $password;
```

protected

Accessible within the class and its subclasses.

```
protected $data;
```

42. Encapsulation

Encapsulation means keeping data and related behavior together while controlling access to internal details.

Example:

```
class BankAccount
{
    private $balance;

    public function deposit($amount)
    {
        $this->balance += $amount;
    }
}
```

The balance is private, so outside code cannot directly modify it.

This protects the object's internal state.

43. Inheritance

Inheritance allows one class to inherit properties and methods from another class.

```
class User
{
```

```
        public function login()
        {
            return "Logged in";
        }
    }

    class Admin extends User
    {
    }
}
```

Now:

```
$admin = new Admin();

$admin->login();
```

works because Admin inherits login() from User.

44. Parent and Child Classes

In:

```
class Admin extends User
User → Parent class
Admin → Child class
```

45. Polymorphism

Polymorphism means that different classes can provide different implementations of the same interface or method contract.

For example, different notification classes could implement:

```
send()
```

but each could send notifications differently.

The calling code can work with the common contract without needing to know the exact implementation.

46. Abstraction

Abstraction means exposing the essential behavior while hiding implementation details.

PHP supports abstraction using abstract classes and interfaces.

Example:

```
abstract class Payment
{
    abstract public function pay();
}
```

A child class must provide the implementation.

47. Interface

An interface defines a contract that implementing classes must follow.

Example:

```
interface Notification
{
    public function send();
}
```

A class implementing it must provide `send()`.

48. Trait

A trait is a mechanism for reusing methods across unrelated classes.

Example:

```
trait Loggable
{
    public function log()
    {
        return "Logging";
    }
}
```

Use it:

```
class Student
```

```
{  
    use Loggable;  
}
```

Traits are useful when multiple classes need the same functionality without requiring inheritance.

49. Namespace

Namespaces organize classes and prevent naming conflicts.

Example:

```
namespace App\Models;
```

A Laravel model may begin with:

```
namespace App\Models;
```

Then it can be imported elsewhere:

```
use App\Models\Item;
```

This tells PHP which class is being referenced.

50. Why Is OOP Important in Laravel?

Laravel itself is built around classes and objects.

Examples:

```
Models  
Controllers  
Middleware  
Requests  
Resources  
Commands  
Policies
```

are commonly implemented as PHP classes.

Therefore, understanding OOP makes Laravel much easier to understand.

DAY 4 — LARAVEL FUNDAMENTALS

51. What Is Laravel?

Laravel is a modern open-source PHP web application framework.

It provides tools and conventions for building web applications efficiently.

Laravel includes features such as:

- Routing
- MVC architecture
- Controllers
- Blade
- Eloquent ORM
- Database migrations
- Validation
- Authentication
- Middleware
- Queues
- Events
- API development
- File storage

52. What Is a Framework?

A framework is a structured foundation that provides reusable functionality and conventions for building applications.

Instead of implementing everything from scratch, developers use the framework's existing tools.

For example, Laravel already provides mechanisms for:

Routing
Validation
Database access
Authentication
Middleware
Views

53. Why Laravel?

Laravel provides:

Developer productivity

Many common tasks are already implemented.

Organization

Laravel encourages a clear project structure.

Security features

Laravel provides mechanisms for:

- CSRF protection
- Password hashing
- Authentication
- Validation
- Secure database interaction

Database support

Laravel provides:

- Migrations
- Eloquent
- Query Builder
- Seeders
- Factories

API support

Laravel supports REST API development and authentication tools such as Sanctum.

54. Laravel Installation

A Laravel project is commonly created through Composer or Laravel's installer.

Example:

```
composer create-project laravel/laravel example-app
```

Then move into the project:

```
cd example-app
```

Run the application:

```
php artisan serve
```

The development server then provides a local URL.

55. Laravel Project Structure

A Laravel application contains many directories.

The most important ones for beginners are:

```
app/  
bootstrap/  
config/  
database/  
public/  
resources/  
routes/  
storage/  
tests/  
vendor/  
.env  
artisan  
composer.json
```

56. app/

Contains the main application code.

Important subdirectories include:

```
app/Models  
app/Http/Controllers  
app/Http/Middleware
```

57. app/Models

Contains Eloquent models.

Example:

```
app/Models/Item.php
```

A model generally represents a database entity/table.

58. app/Http/Controllers

Contains controllers.

Example:

```
app/Http/Controllers/ItemController.php
```

Controllers handle requests and application flow.

59. routes/

Contains route definitions.

Important route files include:

```
routes/web.php  
routes/api.php
```

`web.php` is generally used for web routes.

`api.php` is used for API routes when API routing is enabled/configured in the application.

60. resources/

Contains frontend resources.

Important directories:

```
resources/views
```

```
resources/css
resources/js
```

Blade templates are normally placed under:

```
resources/views
```

61. database/

Contains database-related code.

Important directories include:

```
database/migrations
database/seeders
database/factories
```

62. public/

Contains files intended to be publicly accessible, including the application's front controller:

```
public/index.php
```

Static assets may also be exposed from this directory depending on the project setup.

63. storage/

Contains generated and stored application files.

It can contain:

- Logs
 - Cached files
 - Uploaded files
 - Framework-generated files
-

64. vendor/

The `vendor` directory contains Composer-installed dependencies.

It is generated by Composer.

Normally, developers do not manually edit package files inside `vendor`.

65. composer.json

Contains Composer project configuration and dependencies.

Example dependency categories can include:

```
Laravel framework
Development packages
Autoload configuration
```

66. .env

`.env` contains environment-specific configuration.

Examples:

```
APP_NAME=Foundly
APP_ENV=local
APP_DEBUG=true

DB_CONNECTION=mysql
DB_DATABASE=foundly
DB_USERNAME=root
DB_PASSWORD=
```

The `.env` file should not be committed publicly when it contains secrets or environment-specific credentials.

67. Configuration vs Environment

The `.env` file contains environment-specific values.

The `config/` directory contains Laravel's configuration files.

Laravel can read environment values through configuration.

This allows the same application code to run in different environments.

For example:

```
Development
Testing
Production
```

can have different database credentials.

68. Artisan

Artisan is Laravel's command-line interface.

It is used for many development and maintenance tasks.

69. Important Artisan Commands

Start development server

```
php artisan serve
```

Show routes

```
php artisan route:list
```

Create model

```
php artisan make:model Item
```

Create controller

```
php artisan make:controller ItemController
```

Create migration

```
php artisan make:migration create_items_table
```

Run migrations

```
php artisan migrate
```

Roll back migrations

```
php artisan migrate:rollback
```

Clear application cache

```
php artisan optimize:clear
```

This can clear several framework caches and is useful when configuration/routes/views appear stale during development.

70. Why Artisan Is Useful

Instead of manually creating Laravel files and structures, Artisan can generate them consistently.

For example:

```
php artisan make:model Item -m
```

can create a model and migration together.

This improves:

- Speed
 - Consistency
 - Productivity
 - Project organization
-

DAY 5 — ROUTING, CONTROLLERS AND BLADE

71. What Is Routing?

Routing determines what should happen when a user accesses a particular URL.

A route connects:

```
HTTP Request
  ↓
URL
  ↓
Application action
```

Example:

```
Route::get('/items', function () {
    return "Items";
});
```

When the user visits:

```
/items
```

Laravel executes the associated action.

72. Basic Route

```
Route::get('/about', function () {
    return view('about');
});
```

This means:

When a GET request is made to `/about`, return the `about` view.

73. Route with Controller

In a real application, logic is generally moved into controllers.

```
Route::get('/items', [ItemController::class, 'index']);
```

This means:

```
/items
  ↓
ItemController
  ↓
index()
```

74. Route Parameters

A route can contain dynamic parameters.

```
Route::get('/items/{id}', function ($id) {
    return $id;
});
```

If the user visits:

```
/items/5
```

then:

```
$id = 5
```

75. Required and Optional Parameters

Required parameter:

```
/items/{id}
```

Optional parameter:

```
/items/{id?}
```

The optional parameter may be omitted.

76. Named Routes

Routes can have names.

```
Route::get('/items', [ItemController::class, 'index'])
    ->name('items.index');
```

Then Blade can generate the URL using:

```
{{ route('items.index') }}
```

Named routes are useful because links do not have to hard-code URLs everywhere.

If the URL later changes, the route name can remain the same.

77. Route Groups

Routes can be grouped to share configuration such as middleware or prefixes.

Example:

```
Route::middleware('auth')->group(function () {

    Route::get('/dashboard', ...);
    Route::get('/profile', ...);

});
```

This means both routes require authentication.

78. Controllers

A Controller is a PHP class that handles application logic associated with requests.

Example:

```
class ItemController extends Controller
{
    public function index()
    {
        return view('items.index');
    }
}
```

79. Controller Method

A controller method represents an action.

Example:

```
public function index()
{
    return view('items.index');
}
```

The `index()` method may be responsible for displaying a list of items.

80. Why Use Controllers?

Controllers help keep route files clean.

Instead of:

```
Route::get('/items', function () {
    // lots of application logic here
});
```

we can use:

```
Route::get('/items', [ItemController::class, 'index']);
```

and place the logic inside:

```
ItemController
```

This improves organization and maintainability.

81. Blade Templates

Blade is Laravel's templating engine.

Blade files usually use:

`.blade.php`

extension.

Example:

`resources/views/items/index.blade.php`

Blade allows developers to combine HTML with Laravel-friendly template syntax.

82. Displaying Data in Blade

Blade uses:

```
{{ $name }}
```

to display escaped output.

Example:

```
<h1>{{ $name }}</h1>
```

If:

```
$name = "Saleha";
```

the browser displays:

```
Saleha
```

Escaped output is important because it helps protect against XSS when displaying untrusted content.

83. Blade Conditions

```
@if($item->type === 'lost')
    <p>Lost Item</p>
@else
    <p>Found Item</p>
@endif
```

84. Blade Loops

```
@foreach($items as $item)
    <h2>{{ $item->name }}</h2>
@endforeach
```

This is commonly used to display database records.

85. Passing Data from Controller to View

Controller:

```
public function index()
{
    $items = Item::all();

    return view('items.index', compact('items'));
}
```

The controller sends `$items` to the view.

Blade:

```
@foreach($items as $item)
    <h2>{{ $item->name }}</h2>
@endforeach
```

86. Another Way to Pass Data

You can also use an associative array:

```
return view('items.index', [
    'items' => $items
]);
```

Both approaches pass data to the view.

87. Complete Laravel Request Flow

This is one of the most valuable diagrams to understand for the viva.

Suppose the user visits:

```
/items
```

Step 1 — Browser

The browser sends:

```
GET /items
```

Step 2 — Route

Laravel finds:

```
Route::get('/items', [ItemController::class, 'index']);
```

Step 3 — Controller

Laravel calls:

```
ItemController::index()
```

Step 4 — Model

The controller may call:

```
$items = Item::all();
```

Step 5 — Database

Eloquent retrieves records from the database.

Step 6 — View

The controller returns:

```
return view('items.index', compact('items'));
```

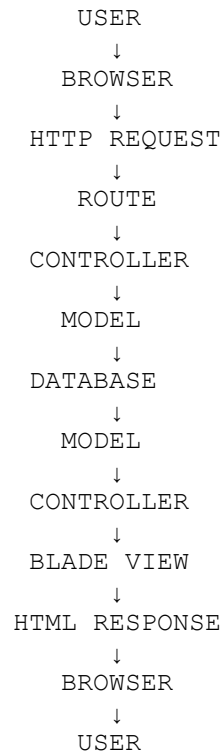
Step 7 — Blade

Blade generates HTML.

Step 8 — Browser

The HTML is returned to the browser and displayed to the user.

Complete flow:



WEEK 1 PRACTICAL UNDERSTANDING

A simple Laravel application can be understood through this example.

Suppose you want to create an Items page for Foundly.

Step 1 — Route

```
Route::get('/items', [ItemController::class, 'index']);
```

Step 2 — Controller

```
public function index()
{
    $items = Item::all();

    return view('items.index', compact('items'));
}
```

Step 3 — Model

```
class Item extends Model
{
}
```

The model represents the `items` table.

Step 4 — View

```
<h1>Foundly Items</h1>

@foreach($items as $item)
    <div>
        <h2>{{ $item->name }}</h2>
        <p>{{ $item->description }}</p>
    </div>
@endforeach
```

Step 5 — User

The user visits:

```
/items
```

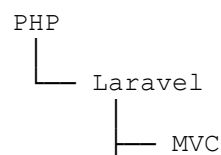
and sees the database records.

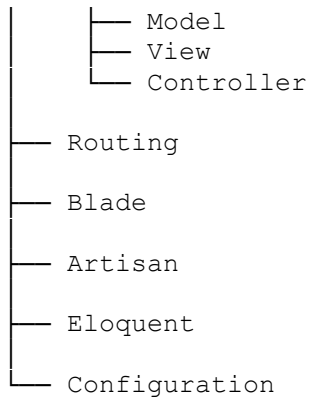
This small example demonstrates:

```
Routing
+
Controller
+
Model
+
Database
+
Blade
+
MVC
```

WEEK 1 — KEY CONCEPT MAP

Memorize this conceptual map:





IMPORTANT TERMINOLOGY

PHP

Server-side programming language used by Laravel.

Laravel

PHP framework used to build web applications.

Framework

A structured foundation that provides reusable functionality and conventions.

Composer

PHP dependency manager.

Artisan

Laravel command-line interface.

MVC

Model, View and Controller architectural pattern.

Model

Represents application data and interacts with the database.

View

User interface/presentation layer.

Controller

Handles request-related application logic.

Route

Maps a URL and HTTP method to an application action.

Blade

Laravel's templating engine.

XAMPP

Local development environment commonly providing Apache, PHP, MySQL/MariaDB and phpMyAdmin.

Apache

Web server.

MySQL/MariaDB

Relational database management system.

phpMyAdmin

Web interface for managing MySQL/MariaDB databases.

HTTP

Protocol for communication between clients and web servers.

HTTPS

Secure version of HTTP.

GET

Retrieve data.

POST

Create/submit data.

PUT

Replace/update a resource.

PATCH

Partially update a resource.

DELETE

Delete a resource.

JSON

Common data-interchange format, especially for APIs.

Authentication

Verifying identity.

Authorization

Determining permissions.

WEEK 1 VIVA QUESTIONS & MODEL ANSWERS

Basic Web Development

Q1. What is web development?

Answer:

Web development is the process of creating and maintaining websites and web applications. It includes frontend development, backend development, database management, testing, and deployment.

Q2. What is the difference between frontend and backend?

Answer:

Frontend is the user-facing part of an application, usually involving HTML, CSS and JavaScript. Backend runs on the server and handles business logic, data processing, authentication and database operations. Laravel and PHP are used on the backend.

Q3. What is client-server architecture?

Answer:

Client-server architecture is a model in which the client sends requests to a server, and the server processes those requests and returns responses. In a web application, the browser normally acts as the client and the Laravel application acts as the server-side application.

Q4. What is a request?

Answer:

A request is a message sent by a client to a server asking it to perform an operation or return a resource.

Q5. What is a response?

Answer:

A response is the server's reply to a client request. It can contain data such as HTML or JSON along with headers and an HTTP status code.

HTTP / HTTPS

Q6. What is HTTP?

Answer:

HTTP stands for HyperText Transfer Protocol. It is a protocol used for communication between web clients and servers.

Q7. What is HTTPS?

Answer:

HTTPS is the secure version of HTTP. It uses encryption through TLS to protect communication between the client and server.

Q8. What is the difference between HTTP and HTTPS?

Answer:

HTTP sends communication without transport encryption, while HTTPS encrypts the communication using TLS. HTTPS is therefore appropriate for protecting sensitive information such as passwords and authentication tokens.

Q9. What are HTTP methods?

Answer:

HTTP methods specify the intended operation of a request. Common methods are GET for retrieving data, POST for creating or submitting data, PUT and PATCH for updating resources, and DELETE for deleting resources.

Q10. What does 404 mean?

Answer:

HTTP 404 means that the requested resource could not be found.

Q11. What is the difference between 401 and 403?

Answer:

401 means the request is not authenticated. 403 means the client is authenticated but does not have permission to perform the requested action.

MVC

Q12. What is MVC?

Answer:

MVC stands for Model-View-Controller. It separates an application into three main responsibilities: the Model handles data, the View handles presentation, and the Controller handles application logic and coordinates requests.

Q13. What is a Model?

Answer:

A Model represents application data and commonly interacts with the database. In Laravel, Eloquent models provide an object-oriented way to work with database records.

Q14. What is a View?

Answer:

A View is responsible for presenting information to the user. Laravel commonly uses Blade templates for views.

Q15. What is a Controller?

Answer:

A Controller is a PHP class that handles application logic associated with HTTP requests. It can retrieve data through models and return a view or another response.

Q16. Why do we use MVC?

Answer:

MVC separates responsibilities, which makes the application easier to organize, maintain, test and modify.

PHP

Q17. What is PHP?

Answer:

PHP is a server-side scripting and programming language widely used for web development. Laravel is built using PHP.

Q18. What is a variable in PHP?

Answer:

A variable stores a value that can be used during program execution. PHP variables begin with the \$ symbol.

Example:

```
$name = "Saleha";
```

Q19. Name some PHP data types.

Answer:

Common PHP data types include string, integer, float, boolean, array, object and null.

Q20. What is the difference between == and ===?

Answer:

== compares values with type conversion where applicable, while === compares both value and type.

Q21. What is a function?

Answer:

A function is a reusable block of code designed to perform a particular task. It can accept parameters and return a value.

Q22. What is an array?

Answer:

An array is a data structure that stores multiple values in a single variable. PHP supports indexed and associative arrays.

OOP

Q23. What is OOP?

Answer:

OOP stands for Object-Oriented Programming. It organizes software around classes and objects and provides concepts such as encapsulation, inheritance, abstraction and polymorphism.

Q24. What is a class?

Answer:

A class is a blueprint that defines the properties and methods that objects created from it can have.

Q25. What is an object?**Answer:**

An object is an instance of a class.

Q26. What is inheritance?**Answer:**

Inheritance allows a child class to reuse and extend properties and methods of a parent class.

Q27. What is encapsulation?**Answer:**

Encapsulation is the practice of keeping data and related behavior together while controlling access to the internal state of an object.

Q28. What is a trait?**Answer:**

A trait is a PHP mechanism for reusing methods across multiple classes without requiring those classes to inherit from the same parent class.

Q29. What is a namespace?**Answer:**

A namespace organizes PHP classes and prevents naming conflicts between classes with the same name.

LARAVEL

Q30. What is Laravel?

Answer:

Laravel is a PHP web application framework that provides tools and conventions for building maintainable web applications. It includes routing, MVC structure, Blade, Eloquent ORM, migrations, validation, middleware, authentication and API support.

Q31. What is a framework?

Answer:

A framework is a software foundation that provides reusable functionality, structure and conventions for developing applications.

Q32. Why use Laravel instead of plain PHP?

Answer:

Laravel provides built-in solutions for common web development tasks such as routing, database access, validation, authentication, middleware and templating. This improves development speed, organization and maintainability.

Q33. What is Artisan?

Answer:

Artisan is Laravel's command-line interface. It is used to perform tasks such as creating models and controllers, running migrations, displaying routes and managing the application.

Q34. What is Composer?

Answer:

Composer is the dependency manager for PHP. Laravel uses it to install and manage PHP packages and project dependencies.

Q35. What is `.env`?

Answer:

`.env` is an environment configuration file containing values such as application settings and database credentials. It allows environment-specific configuration without hard-coding those values into application code.

Q36. What is `composer.json`?

Answer:

`composer.json` defines a PHP project's Composer dependencies and configuration.

Q37. What is `vendor`?

Answer:

The `vendor` directory contains Composer-installed PHP dependencies. It is generated by Composer.

ROUTING

Q38. What is a route?

Answer:

A route maps an incoming HTTP request, usually based on its method and URL, to an application action such as a controller method.

Q39. Give an example of a Laravel route.

Answer:

```
Route::get('/items', [ItemController::class, 'index']);
```

This maps a GET request for `/items` to the `index()` method of `ItemController`.

Q40. What is a route parameter?

Answer:

A route parameter is a dynamic value included in a route URL.

Example:

```
Route::get('/items/{id}', ...);
```

If the user visits `/items/5`, the parameter value is 5.

Q41. What is a named route?

Answer:

A named route is a route assigned a name that can be referenced elsewhere in the application.

Example:

```
->name('items.index');
```

It can then be used with:

```
route('items.index')
```

BLADE

Q42. What is Blade?

Answer:

Blade is Laravel's templating engine. It allows developers to create dynamic HTML views using Blade directives and PHP data.

Q43. What is a Blade file?

Answer:

A Blade template is usually stored in a `.blade.php` file under `resources/views`.

Q44. How do you display a variable in Blade?

Answer:

```
{{ $name }}
```

The `{{ }}` syntax displays escaped output.

Q45. How do you loop through data in Blade?

Answer:

```
@foreach($items as $item)
    {{ $item->name }}
@endforeach
```

Q46. How do you pass data from a controller to a view?

Answer:

For example:

```
$items = Item::all();
return view('items.index', compact('items'));
```

The `$items` variable is then available inside the Blade view.

FINAL WEEK 1 VIVA SCENARIO

Examiner:

"Explain what happens when a user opens `/items` in your Laravel application."

Strong Answer:

"First, the browser sends an HTTP GET request to the Laravel application. Laravel checks the registered routes and finds the route corresponding to `/items`. That route points to the `index` method of the `ItemController`. The controller can use the `Item` Eloquent model to retrieve records from the database. The retrieved data is then passed to a Blade view. Blade generates the HTML response, which Laravel sends back to the browser and the user sees the items page."

This answer demonstrates that you understand:

```
HTTP
↓
Routing
↓
Controller
↓
Model
↓
Database
↓
Blade
↓
Response
```

RAPID-FIRE REVISION

Before entering the viva, remember these:

```
PHP
→ Programming language

Laravel
→ PHP framework

Composer
→ PHP dependency manager

Artisan
→ Laravel CLI

MVC
→ Model + View + Controller

Model
→ Data/database

View
→ User interface
```

Controller

→ Application/request logic

Route

→ Maps request to an action

Blade

→ Laravel templating engine

XAMPP

→ Local PHP development environment

Apache

→ Web server

MySQL/MariaDB

→ Database system

phpMyAdmin

→ Database management interface

GET

→ Retrieve

POST

→ Create/submit

PUT

→ Replace/update

PATCH

→ Partial update

DELETE

→ Delete

200

→ Success

201

→ Created

401

→ Unauthenticated

403

→ Forbidden

404

→ Not Found

422

→ Validation error

500

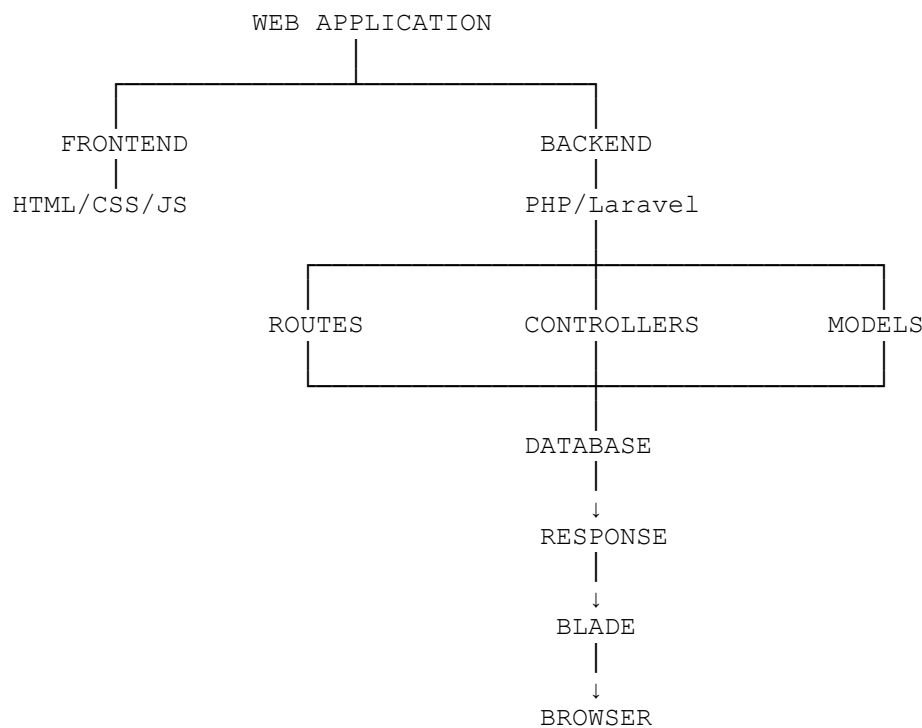
→ Server error

Authentication
→ Who are you?

Authorization
→ What can you do?

THE MOST IMPORTANT CONCEPTUAL DIAGRAM

If you remember only one diagram from Week 1, remember this:



The entire purpose of Week 1 is to understand this flow.

Once you understand this, the database, CRUD, authentication, relationships and API topics in Weeks 2–4 become much easier because they are extensions of the same architecture.

Laravel Internship Viva Handbook — Week 2

Database & CRUD Development

Week 2 is where Laravel starts **talking to a database**.

The key idea is:

Week 1: We learned how a Laravel application receives requests and displays pages.

Week 2: We learn how Laravel stores, retrieves, validates, updates, and deletes real data.

The overall flow becomes:

```
Browser
  ↓
Route
  ↓
Controller
  ↓
Model / Eloquent
  ↓
Database
  ↓
Model / Eloquent
  ↓
Controller
  ↓
Blade View
  ↓
Browser
```

DAY 6 — Migrations & Database Configuration

1. What Is a Database?

A **database** is an organized collection of data that an application can store, retrieve, update, and delete.

For example, your Foundly application might have:

```
users
items
categories
claim_requests
```

A database allows the application to permanently store information rather than losing it whenever the application stops running.

2. Database vs Table vs Record vs Column

These four terms are important.

Database

A container holding related tables.

Example:

```
foundly
```

Table

A structured collection of related records.

Example:

```
items
```

Record / Row

One individual entry in a table.

Example:

```
1 | Wallet | Lost | Library
```

Column

A specific attribute of every record.

Example:

```
id  
name  
description  
location  
type
```

So:

```
Database  
  ↓  
items table  
  ↓  
Rows / Records
```

↓
Columns

3. Relational Database

Laravel commonly works with relational databases such as:

- MySQL
- MariaDB
- PostgreSQL
- SQLite

A relational database stores information in tables and establishes relationships between those tables.

For example:

```
categories
  ↓
items
```

An item can belong to a category.

4. Database Configuration in Laravel

Laravel gets database configuration from the `.env` file.

A typical MySQL configuration looks like:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=foundly
DB_USERNAME=root
DB_PASSWORD=
```

Meaning

Setting	Meaning
DB_CONNECTION	Database type
DB_HOST	Database server address
DB_PORT	Database server port

Setting	Meaning
DB_DATABASE	Database name
DB_USERNAME	Database username
DB_PASSWORD	Database password

5. Why Use `.env` for Database Credentials?

We don't want database credentials hard-coded throughout our application.

Instead:

```
.env
↓
Laravel configuration
↓
Database connection
```

This also allows different environments to use different databases.

For example:

```
Development → local database
Testing → testing database
Production → production database
```

without changing the application's source code.

6. What Is a Migration?

A **migration** is a PHP file that describes a change to the database structure.

Think of migrations as:

Version control for your database schema.

Instead of manually creating tables through phpMyAdmin, you can define the table structure in code.

7. Why Are Migrations Important?

Suppose you manually create an `items` table on your computer.

Now another developer joins the project.

How will they know:

- which columns you created?
- which types you used?
- which keys exist?
- which constraints exist?

A migration provides that structure in a repeatable form.

8. Creating a Migration

Artisan command:

```
php artisan make:migration create_items_table
```

Laravel creates a migration file under:

```
database/migrations
```

9. Basic Migration Structure

Example:

```
Schema::create('items', function (Blueprint $table) {  
    $table->id();  
    $table->string('name');  
    $table->text('description');  
    $table->string('type');  
    $table->string('location');  
    $table->timestamps();  
});
```

10. Understanding Migration Columns

```
$table->id()
```

Creates an auto-incrementing primary key.

Example:

1
2
3
4

`$table->string()`

Creates a string/varchar-type column.

```
$table->string('name');
```

`$table->text()`

Used for longer text.

```
$table->text('description');
```

`$table->integer()`

Creates an integer column.

```
$table->integer('age');
```

`$table->boolean()`

Stores true/false values.

```
$table->boolean('is_active');
```

`$table->date()`

Stores a date.

```
$table->date('date');
```

`$table->dateTime()`

Stores date and time.

```
$table->dateTime('date');
```

```
$table->timestamps()
```

Creates:

```
created_at
updated_at
```

Laravel uses these columns automatically for Eloquent models by default.

11. Primary Key

A primary key uniquely identifies a record.

Usually:

```
id
```

Example:

```
id | name
---|-----
1  | Wallet
2  | Phone
3  | Keys
```

No two records should have the same primary key.

12. Foreign Key

A foreign key connects one table to another.

Suppose:

```
categories
id | name
1  | Electronics
2  | Documents
```

and:

```
items
id | name   | category_id
1  | Phone  | 1
```

2 | Wallet | 2

`category_id` is a foreign key referencing the category.

Conceptually:

```
categories.id
      ↑
      |
items.category_id
```

13. Running Migrations

```
php artisan migrate
```

Laravel reads pending migrations and applies them to the database.

14. Rolling Back Migrations

```
php artisan migrate:rollback
```

This rolls back the latest migration batch.

15. Migration vs Database Seeder

Don't confuse these.

Migration

Defines **database structure**.

Example:

Create an `items` table.

Seeder

Inserts **sample/default data**.

Example:

Add 10 sample items.

So:

Migration → Structure
Seeder → Data

DAY 7 — MODELS & ELOQUENT ORM

16. What Is a Model?

A Laravel Model represents an application entity and usually corresponds to a database table.

Example:

```
Item model
  ↓
items table
```

Model:

```
app/Models/Item.php
```

17. Creating a Model

```
php artisan make:model Item
```

You can also create a model and migration together:

```
php artisan make:model Item -m
```

18. Eloquent ORM

Eloquent is Laravel's **Object-Relational Mapper (ORM)**.

It allows developers to work with database records using PHP models and methods.

Instead of always writing:

```
SELECT * FROM items;
```

you can write:

```
$items = Item::all();
```

Eloquent translates the model operation into appropriate database queries.

19. Why Is Eloquent Useful?

It makes database interaction:

- More readable
 - Object-oriented
 - Easier to maintain
 - Integrated with Laravel relationships
 - Convenient for CRUD operations
-

20. Retrieving All Records

```
$items = Item::all();
```

This retrieves all items.

21. Finding One Record

```
$item = Item::find(1);
```

This searches for the record whose primary key is 1.

22. `findOrFail()`

```
$item = Item::findOrFail($id);
```

If the record doesn't exist, Laravel throws a model-not-found exception, which normally results in a 404 response.

This is safer than assuming the record exists.

23. `where()`

You can filter records:

```
$items = Item::where('type', 'lost')->get();
```

Meaning:

Get items whose type is `lost`.

24. `first()`

```
$item = Item::where('name', 'Wallet')->first();
```

Returns the first matching record or `null`.

25. `get()`

```
$items = Item::where('type', 'lost')->get();
```

`get()` executes the query and returns the matching records as an Eloquent collection.

26. `latest()`

```
$items = Item::latest()->get();
```

This generally orders records by the latest `created_at` timestamp.

This is useful when you want recently added records first.

27. Eloquent Collection

When multiple records are returned, Eloquent commonly returns a **Collection**.

Example:

```
$items = Item::all();
```

You can then use collection methods such as:

```
$items->count();
```

or loop through them:

```
foreach ($items as $item) {  
    echo $item->name;  
}
```

28. Creating Data with Eloquent

One approach:

```
$item = new Item();  
  
$item->name = "Wallet";  
$item->description = "Black wallet";  
$item->type = "lost";  
  
$item->save();
```

`save()` persists the model to the database.

29. Mass Assignment

Laravel also allows:

```
$item = Item::create([  
    'name' => 'Wallet',  
    'description' => 'Black wallet',  
    'type' => 'lost',  
]);
```

However, Laravel requires the relevant fields to be allowed for mass assignment.

Depending on your Laravel version/configuration, this may be managed through model configuration such as `$fillable` or `$guarded`.

Example:

```
protected $fillable = [  
    'name',  
    'description',  
    'type',  
];
```

30. Updating Data

```
$item = Item::find(1);  
  
$item->name = "Black Wallet";  
$item->save();
```

Or:

```
$item->update([  
    'name' => 'Black Wallet',  
]);
```

31. Deleting Data

```
$item = Item::find(1);  
  
$item->delete();
```

The corresponding database record is deleted.

32. Query Builder vs Eloquent

Laravel provides both.

Query Builder

```
DB::table('items')->get();
```

Eloquent

```
Item::all();
```

Eloquent works through models and is especially useful when your application has relationships and model behavior.

DAY 8 — FORMS, VALIDATION & REQUESTS

33. What Is an HTML Form?

A form allows users to submit information.

Example:

```
<form method="POST" action="/items">
    ...
</form>
```

A form may contain:

- Text input
 - Email
 - Password
 - Select
 - Radio buttons
 - Checkboxes
 - File uploads
 - Textareas
-

34. GET vs POST Forms

A GET form is generally used for retrieving/filtering data.

Example:

```
<form method="GET">
```

A POST form is generally used for submitting data that changes server-side state.

Example:

```
<form method="POST">
```

35. CSRF Protection

Laravel protects web forms against **Cross-Site Request Forgery (CSRF)**.

In a Blade form, you commonly write:

```
@csrf
```

Laravel generates a CSRF token.

The token helps verify that a state-changing request came from a legitimate form/session rather than an unauthorized third-party site.

36. What Is a Request?

Laravel's `Request` object represents the incoming HTTP request.

Example:

```
use Illuminate\Http\Request;

public function store(Request $request)
{
    $name = $request->name;
}
```

The request can contain:

- Form data
 - Query parameters
 - Uploaded files
 - Headers
 - Cookies
 - Authentication-related information
-

37. Accessing Form Data

If a form sends:

```
name = Wallet
```

you can access it through:

```
$request->name
```

or:

```
$request->input('name')
```

38. What Is Validation?

Validation checks whether submitted data satisfies defined rules.

Example:

```
$request->validate([
    'name' => 'required|string|max:255',
    'description' => 'required',
]);
```

This ensures:

- name is required
 - name must be a string
 - name cannot exceed 255 characters
 - description is required
-

39. Why Is Validation Important?

Validation protects application data from incorrect or incomplete input.

Without validation, a user might submit:

```
Name:
Description:
Age: abc
Email: hello
```


Validation ensures the application only accepts appropriate data.

40. Common Validation Rules

required

`required`

Field must be present.

string

`string`

Must be a string.

integer

`integer`

Must be an integer.

numeric

`numeric`

Must be numeric.

email

`email`

Must have a valid email format.

max

`max:255`

Maximum allowed value/length depends on the attribute type.

min

`min:3`

Minimum value/length depending on the attribute type.

unique

```
unique:users,email
```

The value must be unique in the specified table/column.

41. Validation Example

Suppose we have a student registration form:

```
$request->validate([
    'name' => 'required|string|max:100',
    'email' => 'required|email',
    'age' => 'required|integer|min:16',
]);
```

If the input is invalid, Laravel automatically handles the validation failure for a normal web request and makes the validation errors available to the redirected form.

42. Displaying Validation Errors

In Blade:

```
@error('name')
    <span>{{ $message }}</span>
@enderror
```

For a general error display, Laravel also provides access to the `$errors` message bag.

43. Old Form Input

If validation fails, Laravel can retain previous input.

Example:

```
<input
    type="text"
```

```
name="name"
value="{{ old('name') }}"
>
```

This prevents users from having to retype everything after a validation error.

44. Form Request Classes

For more complex applications, validation can be moved into a dedicated Form Request class.

Create one:

```
php artisan make:request StoreItemRequest
```

Then validation rules can be defined in that class.

This keeps controllers cleaner and separates validation logic from application logic.

DAY 9 — COMPLETE CRUD OPERATIONS

45. What Is CRUD?

CRUD stands for:

Create
Read
Update
Delete

These are the four fundamental operations performed on stored data.

46. CREATE

Create means adding a new record.

Example:

```
$item = new Item();  
  
$item->name = $request->name;  
$item->description = $request->description;  
  
$item->save();
```

Flow:

```
Form  
↓  
POST request  
↓  
Controller  
↓  
Validation  
↓  
Model  
↓  
Database
```

47. READ

Read means retrieving existing data.

```
$items = Item::all();
```

or:

```
$item = Item::findOrFail($id);
```

48. UPDATE

Update means changing an existing record.

```
$item = Item::findOrFail($id);  
  
$item->name = $request->name;  
$item->save();
```

49. DELETE

Delete means removing an existing record.

```
$item = Item::findOrFail($id);  
$item->delete();
```

50. Complete CRUD Flow

```
CREATE  
↓  
POST  
↓  
Controller  
↓  
Validate  
↓  
Model  
↓  
Database
```

```
READ  
↓  
GET  
↓  
Controller  
↓  
Model  
↓  
Database  
↓  
View
```

```
UPDATE  
↓  
PUT/PATCH  
↓  
Controller  
↓  
Validate  
↓  
Model  
↓  
Database
```

```
DELETE  
↓  
DELETE  
↓  
Controller  
↓
```

Model
↓
Database

51. Resource Controller

Laravel provides a standard structure for CRUD controllers.

Command:

```
php artisan make:controller ItemController --resource
```

It creates methods such as:

```
index()  
create()  
store()  
show()  
edit()  
update()  
destroy()
```

52. Resource Controller Methods

Method	Purpose
<code>index()</code>	Display list of records
<code>create()</code>	Display create form
<code>store()</code>	Store new record
<code>show()</code>	Display one record
<code>edit()</code>	Display edit form
<code>update()</code>	Update existing record
<code>destroy()</code>	Delete record

Memorize this order:

```
INDEX  
CREATE  
STORE  
SHOW  
EDIT  
UPDATE  
DESTROY
```

53. Resource Routes

Instead of manually defining seven routes, Laravel can generate the conventional CRUD routes:

```
Route::resource('items', ItemController::class);
```

This creates routes corresponding to the resource controller methods.

You can inspect them using:

```
php artisan route:list
```

54. Why Resource Controllers?

They provide a standard, predictable structure for CRUD applications.

Instead of every developer inventing different method names, Laravel follows a common convention.

This improves:

- Consistency
 - Readability
 - Productivity
 - Maintainability
-

55. HTML Forms and PUT/PATCH/DELETE

HTML forms natively support GET and POST.

Laravel allows forms to represent other HTTP methods through method spoofing.

For example:

```
<form method="POST" action="{{ route('items.update', $item) }}">
    @csrf
    @method('PUT')
</form>
```

Similarly, for delete:

```
<form method="POST" action="{{ route('items.destroy', $item) }}">
    @csrf
    @method('DELETE')
</form>
```

DAY 10 — BLADE LAYOUTS, COMPONENTS & UI

56. Why Blade Layouts?

Imagine every page contains:

```
Navbar
Footer
CSS
JavaScript
HTML structure
```

If you copy all of this into every page, the application becomes difficult to maintain.

Blade layouts allow you to define a common structure once.

57. Layout

Example:

```
resources/views/layouts/app.blade.php
```

The layout can contain:

```
<!DOCTYPE html>
<html>
<head>
    ...
</head>

<body>

    <nav>
        ...
    </nav>
```



```
@yield('content')

<footer>
    ...
</footer>

</body>
</html>
```

58. @extends

A child view can extend the layout:

```
@extends('layouts.app')
```

59. @section

The child view provides content:

```
@section('content')

    <h1>Items</h1>

@endsection
```

The layout receives that content through:

```
@yield('content')
```

60. Layout Flow

```
layouts/app.blade.php
    ↑
    |
@extends
    |
items/index.blade.php
    |
@section
    ↓
@yield
```

61. @include

@include allows one Blade file to include another.

Example:

```
@include('partials.navbar')
```

Useful for reusable sections such as:

- Navbar
- Footer
- Alerts
- Sidebar

62. Blade Components

Components are reusable UI building blocks.

For example, Foundly can have:

Item Card

Instead of writing the same item card HTML repeatedly, create a reusable component.

Conceptually:

```
Item Card Component
├── Name
├── Description
├── Category
├── Location
└── Actions
```

Then reuse it for every item.

63. Why Components?

Components provide:

- Reusability

- Consistency
- Cleaner templates
- Easier maintenance

If you change the design of the item card once, every use of that component can benefit from the change.

64. Bootstrap

Bootstrap is a frontend CSS framework.

It provides:

- Grid system
- Buttons
- Cards
- Forms
- Navigation
- Responsive utilities
- Typography
- Layout utilities

Example:

```
<button class="btn btn-primary">  
  Add Item  
</button>
```

65. Responsive Design

Responsive design means the interface adapts to different screen sizes.

For example:

```
Desktop → multiple columns  
Tablet  → fewer columns  
Mobile  → single column
```

Bootstrap provides responsive utilities and grid classes to make this easier.

66. Bootstrap vs Custom CSS

Bootstrap provides pre-built styling and layout utilities.

Custom CSS gives you complete control over the visual design.

In your Foundly project, you also worked with a custom:

```
resources/css/app.css
```

This is useful when you want a specific design rather than relying entirely on Bootstrap's default styles.

WEEK 2 — COMPLETE DATABASE REQUEST FLOW

Now combine everything you've learned.

Suppose a user wants to add a Lost Item.

Step 1 — User opens form

```
GET /items/create
```

Route:

```
Route::get('/items/create', [ItemController::class, 'create']);
```

Controller:

```
public function create()
{
    return view('items.create');
}
```

Blade displays the form.

Step 2 — User submits form

```
POST /items
```

The form contains:

```
@csrf
```

Step 3 — Controller receives Request

```
public function store(Request $request)
{
    ...
}
```

Step 4 — Validation

```
$request->validate([
    'name' => 'required|string|max:255',
    'description' => 'required',
]);
```

Step 5 — Eloquent

```
Item::create([
    'name' => $request->name,
    'description' => $request->description,
]);
```

Step 6 — Database

The record is inserted into:

```
items
```

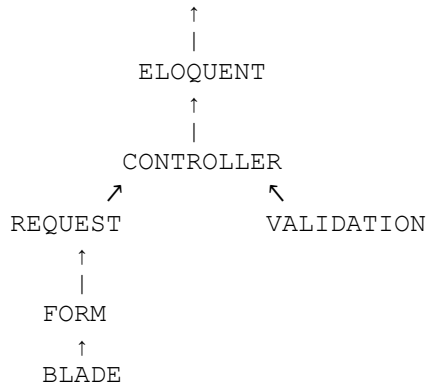
Step 7 — Redirect

The controller may redirect the user back to the item list.

```
return redirect()->route('items.index');
```

WEEK 2 — CONCEPTUAL MAP

DATABASE



This is the central idea of Week 2.

WEEK 2 — IMPORTANT TERMINOLOGY

Database

Organized storage of application data.

Table

Collection of related records.

Row / Record

One entry in a table.

Column

An attribute of a record.

Primary Key

Unique identifier for a record.

Foreign Key

Column used to reference a record in another table.

Migration

Code-based definition of database structure changes.

Model

PHP representation of an application entity/database table.

Eloquent

Laravel's ORM.

ORM

Object-Relational Mapping.

Collection

Laravel's object-oriented wrapper for sets of retrieved records.

Request

Object representing an incoming HTTP request.

Validation

Checking whether submitted data meets defined rules.

CRUD

Create, Read, Update, Delete.

Resource Controller

Controller following Laravel's standard CRUD method structure.

Resource Route

A route definition that creates conventional CRUD routes for a resource.

Blade Layout

Reusable common page structure.

Blade Component

Reusable UI element.

Bootstrap

Frontend CSS/UI framework.

CSRF

Cross-Site Request Forgery; Laravel provides protection for state-changing web requests.

WEEK 2 — VIVA QUESTIONS & MODEL ANSWERS

Database

Q1. What is a database?

Answer:

A database is an organized system for storing and managing application data. A Laravel application can use databases such as MySQL, MariaDB, PostgreSQL or SQLite.

Q2. What is a table?

Answer:

A table is a structured collection of related records organized into rows and columns.

Q3. What is a primary key?

Answer:

A primary key uniquely identifies each record in a table. In Laravel, the default primary key is usually an auto-incrementing `id`.

Q4. What is a foreign key?

Answer:

A foreign key is a column that references a key in another table and is used to establish relationships between tables.

Q5. What is a migration?

Answer:

A migration is a version-controlled PHP file used to define changes to the database schema, such as creating or modifying tables.

Q6. Why do we use migrations?

Answer:

Migrations make database structure changes reproducible, organized and version-controlled, so developers can create the same schema across different environments.

Q7. What is the difference between a migration and a seeder?

Answer:

A migration defines or changes database structure, while a seeder inserts initial, sample or test data into the database.

ELOQUENT

Q8. What is Eloquent?

Answer:

Eloquent is Laravel's ORM. It allows developers to interact with database records through PHP models instead of writing raw SQL for every operation.

Q9. What does ORM mean?

Answer:

ORM stands for Object-Relational Mapping. It maps application objects and classes to relational database records and tables.

Q10. How do you retrieve all records using Eloquent?

Answer:

```
$items = Item::all();
```

Q11. How do you find a record by ID?

Answer:

```
$item = Item::find(1);
```

If I want Laravel to automatically produce a not-found response when it doesn't exist, I can use:

```
$item = Item::findOrFail(1);
```

Q12. How do you filter records?

Answer:

Using `where()`:

```
$items = Item::where('type', 'lost')->get();
```

Q13. How do you insert data using Eloquent?

Answer:

One approach is:

```
$item = new Item();  
$item->name = 'Wallet';  
$item->save();
```

Alternatively, `create()` can be used when mass-assignment configuration allows the fields.

Q14. How do you update a record?

Answer:

```
$item = Item::findOrFail($id);  
  
$item->name = 'Phone';  
$item->save();
```

Q15. How do you delete a record?

Answer:

```
$item = Item::findOrFail($id);  
$item->delete();
```

FORMS & VALIDATION

Q16. What is validation?

Answer:

Validation checks whether user-submitted data satisfies predefined rules before the application processes or stores it.

Q17. Why is validation important?

Answer:

It prevents invalid or incomplete data from entering the application and improves data quality and application reliability.

Q18. What is CSRF protection?

Answer:

CSRF protection helps prevent attackers from tricking an authenticated user's browser into sending unauthorized state-changing requests. Laravel provides CSRF tokens for web forms.

Q19. What does @csrf do?**Answer:**

@csrf generates a hidden CSRF token field in a Blade form. Laravel verifies the token when processing the request.

Q20. What is the Request object?**Answer:**

Laravel's Request object represents the incoming HTTP request and provides access to submitted data, query parameters, headers, uploaded files and other request information.

CRUD

Q21. What is CRUD?**Answer:**

CRUD stands for Create, Read, Update and Delete. These are the four fundamental operations performed on persistent data.

Q22. Explain CRUD in Laravel.**Answer:**

Create inserts a new record, Read retrieves existing records, Update modifies an existing record, and Delete removes a record. Laravel commonly performs these operations through controllers and Eloquent models.

Q23. What is a resource controller?**Answer:**

A resource controller follows Laravel's conventional CRUD method structure: `index`, `create`, `store`, `show`, `edit`, `update`, and `destroy`.

Q24. What does `index()` do?

Answer:

`index()` conventionally displays a list of resources.

Q25. What does `create()` do?

Answer:

`create()` conventionally displays the form used to create a new resource.

Q26. What does `store()` do?

Answer:

`store()` processes the submitted data and creates a new resource.

Q27. What does `show()` do?

Answer:

`show()` displays a specific resource.

Q28. What does `edit()` do?

Answer:

`edit()` displays a form for editing an existing resource.

Q29. What does `update()` do?

Answer:

`update()` processes submitted data and modifies an existing resource.

Q30. What does `destroy()` do?

Answer:

`destroy()` deletes an existing resource.

BLADE

Q31. What is Blade?

Answer:

Blade is Laravel's templating engine. It provides syntax for displaying dynamic data, conditions, loops, layouts and reusable components within HTML views.

Q32. Why use Blade layouts?

Answer:

Layouts allow common elements such as navigation, footer and page structure to be defined once and reused across multiple pages.

Q33. What does `@extends` do?

Answer:

`@extends` allows a Blade view to inherit a layout.

Q34. What does `@section` do?

Answer:

`@section` defines content for a section of the parent layout.

Q35. What does `@include` do?

Answer:

`@include` inserts another Blade view into the current view.

Q36. What is a Blade component?

Answer:

A Blade component is a reusable UI element that helps keep views modular, consistent and maintainable.

IMPORTANT SCENARIO QUESTIONS

Q37. Explain how a form submission works in Laravel.

Answer:

The user fills out a Blade form and submits it using an HTTP request. The route sends the request to a controller method. The controller receives the `Request` object, validates the submitted data, uses an Eloquent model to save the valid data to the database, and then returns a response such as a redirect or view.

Q38. What happens if validation fails?

Answer:

For a normal web request, Laravel redirects back to the form with validation errors and commonly preserves the previously submitted input. The Blade view can display those errors using directives such as `@error` and old values using `old()`.

Q39. Why use Eloquent instead of writing SQL everywhere?

Answer:

Eloquent provides an object-oriented, expressive way to interact with database records and integrates naturally with Laravel models, relationships, collections and other framework features. Raw SQL can still be used when it is appropriate.

Q40. Explain the complete CRUD lifecycle.

Answer:

For creation, the user submits a form through POST, the controller validates the request and uses Eloquent to insert the record. For reading, the controller retrieves records using Eloquent and passes them to a Blade view. For updating, the user opens an edit form and submits a PUT or PATCH request, which the controller validates and updates through Eloquent. For deletion, a DELETE request reaches the controller and the corresponding Eloquent record is deleted.

FOUNDLY-BASED VIVA QUESTIONS

Since your internship project was **Foundly**, these are particularly important.

Q41. How did you use CRUD in Foundly?

Answer:

I used CRUD operations for managing lost and found items. Users could create an item, view existing items, edit item information, and delete items. Laravel controllers handled the operations while Eloquent models interacted with the database.

Q42. What information did an Item contain?

Answer:

An item could contain information such as its name, description, type, location, date and category. The exact fields depend on the application's database design.

Q43. Why did you create an Item model?

Answer:

The Item model represents the item entity in the application and provides an Eloquent interface for interacting with the corresponding `items` database table.

Q44. How did Laravel retrieve items?

Answer:

I could retrieve items through Eloquent, for example:

```
$items = Item::all();
```

or with additional query behavior:

```
$items = Item::latest()->get();
```

Q45. Why would you use `Item::with('category')`?

Answer:

`with('category')` performs eager loading of the category relationship. This allows the application to retrieve the related category data efficiently instead of repeatedly querying it while displaying items.

Q46. How would you validate a Foundly item form?

Answer:

I would define validation rules for required fields and appropriate data types. For example:

```
$request->validate([
    'name' => 'required|string|max:255',
    'description' => 'required|string',
    'type' => 'required|string',
```

```
'location' => 'required|string',  
]);
```

Q47. Why is validation necessary in Foundly?

Answer:

Validation ensures that users cannot submit incomplete or incorrectly formatted item information. It helps maintain consistent and reliable database records.

★ WEEK 2 "EXPLAIN IT LIKE A DEVELOPER" QUESTION

If the examiner says:

"You have an item registration form. Explain everything that happens when I submit it."

Give this answer:

"The user fills out the Blade form and submits it using a POST request. The form includes a CSRF token for protection. The request is matched to the appropriate route and passed to the controller. The controller receives Laravel's Request object and validates the submitted fields. If validation fails, Laravel redirects back with validation errors. If validation succeeds, the controller uses the Item Eloquent model to create and save the record in the database. Finally, the controller can redirect the user to the item listing page with a success message."

That one answer demonstrates your understanding of:

```
Blade  
↓  
HTML Form  
↓  
POST  
↓  
CSRF  
↓  
Route  
↓  
Request  
↓  
Controller  
↓  
Validation  
↓
```

Eloquent
↓
Database
↓
Redirect

WEEK 2 — LAST-MINUTE REVISION SHEET

Memorize these exact associations:

Migration
→ Database structure

Seeder
→ Database data

Model
→ Represents application data/entity

Eloquent
→ Laravel ORM

ORM
→ Object Relational Mapping

Request
→ Incoming HTTP request

Validation
→ Checks submitted data

CSRF
→ Protects state-changing web requests

CRUD
→ Create, Read, Update, Delete

Resource Controller
→ Standard CRUD controller

index()
→ List

create()
→ Create form

store()
→ Save

show()

→ One record

edit()

→ Edit form

update()

→ Update

destroy()

→ Delete

Blade Layout

→ Common page structure

@extends

→ Use a layout

@section

→ Define layout content

@include

→ Include another Blade view

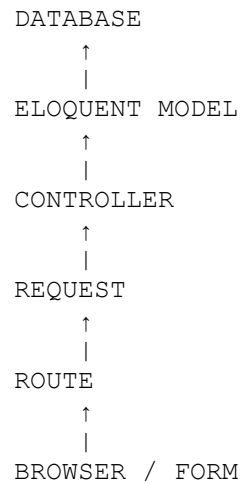
Component

→ Reusable UI

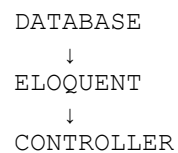
Bootstrap

→ CSS/UI framework

And remember the most important Week 2 flow:



For displaying data, the direction becomes:



↓
BLADE
↓
HTML
↓
BROWSER

If you can confidently explain that flow, plus CRUD, migrations, Eloquent, validation, and resource controllers, you have the core of Week 2 covered.

Absolutely. Below is a **professional, detailed Week 3 + Week 4 handbook-style explanation**. I'll explain each concept as you would need to understand it for **development work, internship documentation, and viva**—including definitions, purpose, syntax, examples, Laravel structure, and how the concepts connect together.

WEEK 3 — INTERMEDIATE LARAVEL

Overall Objective

Week 3 moves from basic CRUD development toward building a **real-world, structured Laravel application**.

By the end of Week 3, you should understand how to:

- Connect related database records using **Eloquent Relationships**
- Authenticate users using **Laravel Breeze**
- Protect routes using **Middleware**
- Control what users are allowed to do using **Authorization**
- Implement **Roles and Permissions**
- Upload and store files/images
- Search and paginate database records
- Use **Resource Controllers**
- Use **Route Model Binding**

These concepts are particularly important in an application such as **Foundly**, because Foundly has users, items, categories, claims, authentication, uploaded images, and different levels of access.

DAY 11 — ELOQUENT RELATIONSHIPS

1. What is a Relationship?

A **database relationship** describes how records in one database table are connected to records in another table.

For example, in Foundly:

```
User
  |
  | has many
  ↓
Items
```

A user can post multiple lost/found items.

Similarly:

```
Category
  |
  | has many
  ↓
Items
```

A category can contain multiple items.

For example:

```
Categories
-----
1 | Electronics
2 | Documents
3 | Accessories
Items
-----
1 | Laptop          | category_id=1
2 | CNIC            | category_id=2
3 | Watch           | category_id=3
4 | Mobile Phone    | category_id=1
```

The `category_id` column creates the connection.

2. Eloquent ORM

Definition

Eloquent ORM is Laravel's built-in Object-Relational Mapper that allows developers to interact with database tables using PHP models instead of writing SQL queries for every operation.

For example, instead of:

```
SELECT * FROM items;
```

you can write:

```
$items = Item::all();
```

Eloquent also provides relationship methods.

```
$user->items
```

This means:

Get the items belonging to this user.

3. Types of Eloquent Relationships

The major relationships are:

1. One-to-One
2. One-to-Many
3. Many-to-Many
4. Has-One-Through
5. Has-Many-Through
6. Polymorphic relationships

For your internship, the most important ones are:

- One-to-One
- One-to-Many
- Many-to-Many

4. One-to-One Relationship

Definition

A **One-to-One relationship** means one record in Table A is associated with exactly one record in Table B.

Example:

User → Profile

One user has one profile.

Model

User:

```
public function profile()  
{  
    return $this->hasOne(Profile::class);  
}
```

Profile:

```
public function user()  
{  
    return $this->belongsTo(User::class);  
}
```

Usage:

```
$user = User::find(1);  
$user->profile;
```

5. One-to-Many Relationship

This is extremely important for Foundly.

Definition

A **One-to-Many relationship** means one record can be associated with many records in another table.

Example:

```
User  
├── Item 1  
├── Item 2  
└── Item 3
```


One user can create many items.

User Model

```
public function items()  
{  
    return $this->hasMany(Item::class);  
}
```

Item Model

```
public function user()  
{  
    return $this->belongsTo(User::class);  
}
```

Now:

```
$user->items;
```

returns all items belonging to the user.

And:

```
$item->user;
```

returns the owner of that item.

6. Foreign Key

Definition

A **foreign key** is a database column that references the primary key of another table.

For example:

```
users  
id
```

and:

```
items  
id  
user_id
```

Here:

```
items.user_id → users.id
```

Migration:

```
$table->foreignId('user_id')
    ->constrained()
    ->onDelete('cascade');
```

`foreignId()` creates a foreign-key-compatible column.

`constrained()` establishes the relationship with the corresponding table.

7. Category Relationship in Foundly

Your Foundly application can have:

```
Category
  ↓
Many Items
```

Category model:

```
public function items()
{
    return $this->hasMany(Item::class);
}
```

Item model:

```
public function category()
{
    return $this->belongsTo(Category::class);
}
```

Then:

```
$item->category
```

gets the item's category.

And:

```
$category->items
```

gets all items in that category.

8. Eager Loading

Definition

Eager loading means loading related records together with the main records to reduce unnecessary database queries.

Instead of:

```
$items = Item::all();
```

you can use:

```
$items = Item::with('category')->get();
```

Now Laravel loads items and their categories efficiently.

You can also load multiple relationships:

```
$items = Item::with(['category', 'user'])->get();
```

9. Why Relationships Matter

Without relationships, you may need to manually write queries to connect records.

With Eloquent:

```
$item->category->name;
```

is enough to access the category name.

This makes Laravel applications:

- Cleaner
 - Easier to maintain
 - More readable
 - More object-oriented
 - Less dependent on raw SQL
-

DAY 12 — AUTHENTICATION WITH LARAVEL BREEZE

1. Authentication

Definition

Authentication is the process of verifying the identity of a user.

In simple terms:

"Who are you?"

Examples:

- Login
- Registration
- Logout
- Password verification
- Password reset

Typical authentication flow:

```
User
↓
Login Form
↓
Email + Password
↓
Laravel verifies credentials
↓
Authenticated User
↓
Dashboard
```

2. Authorization vs Authentication

These two terms are frequently confused.

Authentication

Answers:

Who is the user?

Example:

Saleha is logged in.

Authorization

Answers:

What is this user allowed to do?

Example:

Saleha can edit her own item.
Saleha cannot delete another user's item.
Admin can delete any item.

So:

Authentication = Identity
Authorization = Permission

3. Laravel Breeze

Definition

Laravel Breeze is a lightweight Laravel starter kit that provides the basic authentication scaffolding for an application.

It commonly provides:

- Registration
- Login
- Logout
- Password confirmation
- Password reset functionality
- Authentication routes
- Controllers
- Views
- Authentication middleware

Installation commonly involves:

```
composer require laravel/breeze --dev
```

Then:

```
php artisan breeze:install
```

Depending on the Laravel version and chosen stack, the installation may ask you to select a frontend/authentication setup.

Then:

```
php artisan migrate  
npm install  
npm run build
```

4. Authentication Routes

Breeze creates authentication-related routes such as:

```
/login  
/register  
/logout  
/forgot-password  
/reset-password
```

The exact route setup can vary depending on Laravel/Breeze version and installation choices.

5. Authenticated User

After login, Laravel can identify the currently logged-in user.

For example:

```
auth()->user()
```

returns the current authenticated user.

You can access:

```
auth()->user()->name
```

or:

```
auth()->user()->email
```

You can also check:

```
if (auth()->check()) {  
    // User is logged in  
}
```

6. Guest User

A **guest** is a user who is not authenticated.

You can protect routes using:

```
Route::middleware('auth')->group(function () {  
  
    Route::get('/dashboard', function () {  
        return view('dashboard');  
    });  
  
});
```

Only logged-in users can access these routes.

DAY 13 — MIDDLEWARE, AUTHORIZATION, ROLES & PERMISSIONS

This is one of the most important days for understanding Laravel security.

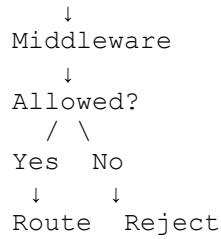
1. Middleware

Definition

Middleware is a mechanism that filters HTTP requests before they reach the application's controller or route.

Think of middleware as a security checkpoint:

Request



2. Authentication Middleware

The most common example:

```
Route::get('/dashboard', function () {  
    return view('dashboard');  
})->middleware('auth');
```

If the user isn't logged in, Laravel prevents access.

3. Middleware Group

You can protect multiple routes:

```
Route::middleware('auth')->group(function () {  
  
    Route::get('/dashboard', ...);  
  
    Route::get('/items', ...);  
  
    Route::post('/items', ...);  
  
});
```

All routes inside the group require authentication.

4. Custom Middleware

You can create middleware:

```
php artisan make:middleware AdminMiddleware
```

This creates:


```
app/  
└─ Http/  
    └─ Middleware/  
        └─ AdminMiddleware.php
```

Example:

```
public function handle($request, Closure $next)  
{  
    if (auth()->user()?->role !== 'admin') {  
        abort(403);  
    }  
  
    return $next($request);  
}
```

Here:

```
abort(403);
```

means:

Forbidden — the user doesn't have permission.

5. Authorization

Definition

Authorization determines whether an authenticated user has permission to perform a particular action.

For example:

```
User A owns Item A
```

User A can:

```
Edit Item A  
Delete Item A
```

But User A should not necessarily be able to:

```
Delete User B's Item
```

6. Gates

A **Gate** is Laravel's mechanism for defining authorization logic.

Example:

```
Gate::define('delete-item', function ($user, $item) {  
    return $user->id === $item->user_id;  
});
```

Then:

```
if (Gate::allows('delete-item', $item)) {  
    // allowed  
}
```

7. Policies

Definition

A **Policy** is a class containing authorization logic for a particular model.

For example:

```
php artisan make:policy ItemPolicy --model=Item
```

Then:

```
public function update(User $user, Item $item)  
{  
    return $user->id === $item->user_id;  
}
```

The policy determines whether a user can update an item.

8. Roles

Definition

A **role** is a group or category assigned to a user that represents their general level of responsibility.

Example:

```
Admin
User
Moderator
```

In Foundly:

```
Admin
  ↓
Manage everything
```

```
User
  ↓
Manage own items
Submit claims
```

A simple database field might be:

```
users
-----
id
name
email
password
role
```

with:

```
role = user
```

or:

```
role = admin
```

9. Permissions

Definition

A **permission** is a specific action that a user is allowed to perform.

Examples:

```
create-item
edit-item
delete-item
approve-claim
manage-users
```

Difference:

Role = What type of user?

Permission = What can they do?

For example:

```
Admin
├── create-item
├── edit-item
├── delete-item
├── approve-claim
└── manage-users
```

DAY 14 — FILE UPLOADS, STORAGE & IMAGE HANDLING

1. File Upload

Definition

A **file upload** allows a user to send a file from their device to the server.

For Foundly, users may upload:

- Item photographs
 - Proof of ownership
 - Identification documents
-

2. HTML File Upload

A form must use:

```
<form method="POST"
      action="/items"
      enctype="multipart/form-data">
```

The important part is:

```
enctype="multipart/form-data"
```

Without this, files generally won't be transmitted correctly.

Input:

```
<input type="file" name="image">
```

3. Laravel Validation

Example:

```
$request->validate([  
    'image' => 'required|image|max:2048',  
]);
```

This means:

```
required → file must exist  
image → must be an image  
max:2048 → maximum size is 2048 KB
```

You can specify formats:

```
'image' => 'required|image|mimes:jpg,jpeg,png,webp|max:2048'
```

4. Laravel Storage

Laravel provides a filesystem abstraction through the `Storage` facade.

Example:

```
use Illuminate\Support\Facades\Storage;
```

Store a file:

```
$path = $request->file('image')->store('items', 'public');
```

This might produce:

```
items/abc123.jpg
```

5. Storage Link

Laravel's public storage usually requires:

```
php artisan storage:link
```

This creates a symbolic link so files stored in:

```
storage/app/public
```

can be accessed through:

```
public/storage
```

6. Displaying an Image

If the stored path is:

```
items/image.jpg
```

you can use:

```
name }}">
```

7. File Replacement

When editing an item, you may want to remove the old image before storing a new one.

Example:

```
if ($request->hasFile('image')) {  
    if ($item->image) {  
        Storage::disk('public')->delete($item->image);  
    }  
  
    $item->image = $request->file('image')  
                ->store('items', 'public');  
}
```

This prevents unnecessary old files from accumulating.

DAY 15 — SEARCH, PAGINATION, RESOURCE CONTROLLERS & ROUTE MODEL BINDING

1. Search

Definition

Search allows users to retrieve database records based on a keyword or condition.

Example:

Search: "wallet"

Laravel query:

```
$items = Item::where('name', 'like', '%' . $request->search . '%')
->get();
```

The % wildcard means other characters can appear before and after the search term.

2. Search Multiple Fields

You can search several columns:

```
$items = Item::where('name', 'like', "%{$search}%")
->orWhere('description', 'like', "%{$search}%")
->orWhere('location', 'like', "%{$search}%")
->get();
```

This can allow users to search by:

Name
Description
Location

3. Pagination

Definition

Pagination divides a large collection of records into smaller pages.

Instead of:

```
1000 items
```

showing everything on one page:

```
Page 1 → 10 items  
Page 2 → 10 items  
Page 3 → 10 items  
...
```

Laravel:

```
$items = Item::paginate(10);
```

Blade:

```
{{ $items->links() }}
```

4. Why Pagination Matters

Pagination:

- Improves performance
 - Reduces database results per request
 - Makes interfaces easier to use
 - Reduces page size
 - Improves user experience
-

5. Resource Controller

Definition

A **Resource Controller** is a Laravel controller designed around the standard CRUD operations.

Create one:

```
php artisan make:controller ItemController --resource
```

Laravel provides methods such as:


```
index()
create()
store()
show()
edit()
update()
destroy()
```

Their purpose:

Method	Purpose
index()	Display records
create()	Display create form
store()	Save new record
show()	Display one record
edit()	Display edit form
update()	Update record
destroy()	Delete record

6. Resource Routes

Instead of manually defining seven routes:

```
Route::get('/items', ...);
Route::get('/items/create', ...);
Route::post('/items', ...);
...
```

you can use:

```
Route::resource('items', ItemController::class);
```

Laravel automatically creates the conventional CRUD routes.

7. Route Model Binding

Definition

Route Model Binding allows Laravel to automatically retrieve a model based on a route parameter.

Without route model binding:

```
Route::get('/items/{id}', function ($id) {  
    $item = Item::findOrFail($id);  
});
```

With route model binding:

```
Route::get('/items/{item}', function (Item $item) {  
    return view('items.show', compact('item'));  
});
```

Laravel automatically retrieves the `Item`.

8. Route Model Binding in Controllers

Example:

```
public function show(Item $item)  
{  
    return view('items.show', compact('item'));  
}
```

Route:

```
Route::get('/items/{item}', [ItemController::class, 'show']);
```

Laravel sees:

```
{item}
```

and:

```
Item $item
```

and automatically retrieves the corresponding model.

WEEK 4 — LARAVEL REST API DEVELOPMENT

Week 4 moves your application from a traditional server-rendered Laravel application toward **API-based application development**.

The central idea is:

```
Client
  ↓
HTTP Request
  ↓
Laravel API
  ↓
Database
  ↓
JSON Response
  ↓
Client
```

The client could be:

- React
- Vue
- Mobile application
- JavaScript frontend
- Postman
- Another server/application

DAY 16 — REST API FUNDAMENTALS

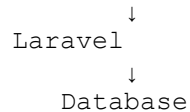
1. API

Definition

API (Application Programming Interface) is a mechanism that allows different software systems to communicate with each other.

For example:

```
React Application
  ↓
  API
```



The frontend doesn't directly access the database.

Instead:

Frontend → Laravel API → Database

2. REST API

Definition

REST (Representational State Transfer) is an architectural style for designing web APIs around resources and standard HTTP operations.

For example, Foundly has an:

`items`

resource.

You can manipulate that resource using HTTP methods.

3. HTTP Methods

The most important HTTP methods are:

Method	Purpose
GET	Retrieve data
POST	Create data
PUT	Replace/update data
PATCH	Partially update data
DELETE	Delete data

Example:

GET	/api/items
POST	/api/items

```
GET      /api/items/5
PUT      /api/items/5
DELETE   /api/items/5
```

4. GET

Used to retrieve information.

```
GET /api/items
```

Response:

```
[
  {
    "id": 1,
    "name": "Wallet"
  },
  {
    "id": 2,
    "name": "Laptop"
  }
]
```

5. POST

Used to create a new resource.

```
POST /api/items
```

Request:

```
{
  "name": "Black Wallet",
  "location": "Library"
}
```

6. PUT

Used to update/replace a resource.

```
PUT /api/items/1
```

Example:

```
{  
  "name": "Brown Wallet",  
  "location": "Cafeteria"  
}
```

7. DELETE

Used to remove a resource.

```
DELETE /api/items/1
```

8. JSON

Definition

JSON (JavaScript Object Notation) is a lightweight data-interchange format commonly used by APIs.

Example:

```
{  
  "id": 1,  
  "name": "Wallet",  
  "location": "Library"  
}
```

JSON is popular because it is:

- Lightweight
 - Human-readable
 - Easy for JavaScript to process
 - Supported by almost every programming language
-

9. API Routes in Laravel

API routes are commonly defined in:

```
routes/api.php
```

Example:

```
Route::get('/items', [ItemController::class, 'index']);
```

Then:

```
GET /api/items
```

can access that route.

10. JSON Response

Laravel:

```
return response()->json([  
    'message' => 'Items retrieved successfully',  
    'data' => $items  
]);
```

Response:

```
{  
    "message": "Items retrieved successfully",  
    "data": []  
}
```

DAY 17 — API CRUD, VALIDATION & POSTMAN

1. API CRUD

CRUD means:

Create
Read
Update
Delete

API CRUD follows the HTTP methods:

Create → POST
Read → GET
Update → PUT/PATCH
Delete → DELETE

2. API Controller

You can create an API controller:

```
php artisan make:controller Api/ItemController --api
```

This is intended for API-style controller actions without HTML-oriented `create()` and `edit()` methods.

3. API Store Method

Example:

```
public function store(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'description' => 'required|string',
        'location' => 'required|string',
    ]);

    $item = Item::create($validated);

    return response()->json([
        'message' => 'Item created successfully',
        'data' => $item
    ], 201);
}
```

4. API Validation

Definition

Validation ensures that incoming data meets predefined requirements before it is stored or processed.

Example:

```
$request->validate([
    'name' => 'required|string|max:255',
    'category_id' => 'required|exists:categories,id',
]);
```


Meaning:

required → value must exist
string → must be text
max:255 → maximum 255 characters
exists → value must exist in database

5. Postman

Definition

Postman is an API development and testing tool used to send HTTP requests and inspect API responses.

You can test:

GET
POST
PUT
PATCH
DELETE

without building a frontend first.

For example:

GET `http://localhost:8000/api/items`

Postman shows the JSON response.

6. Why Postman is Important

Postman allows you to test:

- API endpoints
- Request parameters
- JSON bodies
- Headers
- Authentication tokens
- Validation
- HTTP status codes
- Error responses

DAY 18 — API AUTHENTICATION WITH SANCTUM

1. API Authentication

API authentication verifies the identity of the client/user making an API request.

For example:

```
Client
↓
API Token
↓
Laravel
↓
Is token valid?
↓
Allow/Deny
```

2. Laravel Sanctum

Definition

Laravel Sanctum provides lightweight authentication for SPAs, mobile applications, and token-based APIs.

It can issue personal access tokens that clients send with API requests.

3. API Token

Definition

An **API token** is a secret credential used to identify and authenticate an API client/user.

Example request header:

```
Authorization: Bearer 1|abcdef123456...
```

The client sends this token with protected requests.

4. Creating a Token

A user model can use Sanctum's trait:

```
use Laravel\Sanctum\HasApiTokens;
```

Then:

```
$token = $user->createToken('api-token')->plainTextToken;
```

The token can be returned:

```
return response()->json([  
    'token' => $token  
]);
```

5. Protected API Routes

Example:

```
Route::middleware('auth:sanctum')->group(function () {  
    Route::get('/items', [ItemController::class, 'index']);  
    Route::post('/items', [ItemController::class, 'store']);  
});
```

Now the client must authenticate.

6. Bearer Token

The client sends:

```
Authorization: Bearer YOUR_TOKEN
```

`Bearer` tells Laravel that the token is being supplied as a bearer authentication credential.

DAY 19 — API RESOURCES, RELATIONSHIPS, ERRORS & STATUS CODES

1. API Resource

Definition

An **API Resource** is a Laravel class used to transform models into a consistent JSON representation.

Create one:

```
php artisan make:resource ItemResource
```

Example:

```
class ItemResource extends JsonResource
{
    public function toArray(Request $request)
    {
        return [
            'id' => $this->id,
            'name' => $this->name,
            'description' => $this->description,
            'location' => $this->location,
        ];
    }
}
```

Then:

```
return new ItemResource($item);
```

2. Why API Resources?

Without resources, your API might accidentally expose unnecessary database fields.

A resource lets you control:

What data is returned

How data is named
How relationships are represented
What information is hidden

3. Resource Collection

For multiple records:

```
return ItemResource::collection($items);
```

4. Relationships in API Resources

Suppose an item belongs to a category.

You can return:

```
'category' => new CategoryResource($this->whenLoaded('category')),
```

Then:

```
$items = Item::with('category')->get();  
return ItemResource::collection($items);
```

The API can return:

```
{  
  "id": 1,  
  "name": "Laptop",  
  "category": {  
    "id": 2,  
    "name": "Electronics"  
  }  
}
```

5. HTTP Status Codes

HTTP status codes tell the client what happened.

200 — OK

Request succeeded.

```
GET /api/items  
→ 200
```

201 — Created

A resource was successfully created.

```
POST /api/items  
→ 201
```

204 — No Content

Request succeeded but there is no response body.

Commonly used after deletion.

400 — Bad Request

The request is malformed or invalid.

401 — Unauthorized

The client is not authenticated.

Example:

```
Missing/invalid token
```

403 — Forbidden

The user is authenticated but does not have permission.

```
Logged in ≠ authorized
```

404 — Not Found

Requested resource doesn't exist.

422 — Unprocessable Content

Validation failed.

Example:

```
{  
  "message": "The name field is required.",  
  "errors": {
```

```
    "name": [
      "The name field is required."
    ]
  }
}
```

500 — Internal Server Error

Unexpected server-side error.

6. API Error Handling

A professional API should return predictable errors.

For example:

```
return response()->json([
  'message' => 'Item not found'
], 404);
```

Instead of exposing internal database errors or sensitive information.

7. Clean API Response

A consistent API structure might look like:

```
{
  "success": true,
  "message": "Item retrieved successfully",
  "data": {
    "id": 1,
    "name": "Wallet"
  }
}
```

Error:

```
{
  "success": false,
  "message": "Item not found",
  "data": null
}
```

Consistency makes APIs easier for frontend developers to consume.

DAY 20 — API DOCUMENTATION, TESTING & BEST PRACTICES

1. API Documentation

Definition

API documentation describes how developers can use an API.

It should explain:

- Endpoint
- HTTP method
- URL
- Authentication
- Parameters
- Request body
- Response
- Status codes
- Errors

Example:

```
GET /api/items
```

Description:

```
Returns a list of available items.
```

2. Example Documentation

Endpoint

```
GET /api/items
```

Authentication

```
Bearer Token required
```

Response


```
{  
  "success": true,  
  "data": []  
}
```

3. API Testing

Definition

API testing is the process of verifying that API endpoints behave correctly under different conditions.

You should test:

Successful request

```
GET /api/items  
→ 200
```

Invalid data

```
POST /api/items  
→ 422
```

Unauthenticated request

```
GET /api/items  
→ 401
```

Unauthorized request

```
DELETE /api/items/5  
→ 403
```

Missing resource

```
GET /api/items/999  
→ 404
```

4. Feature Tests

Laravel provides testing support.

Example:

```
$response = $this->getJSON('/api/items');  
  
$response->assertStatus(200);
```

You can test JSON:

```
$response->assertJson([  
    'success' => true  
]);
```

5. API Best Practices

Use meaningful endpoints

Prefer:

```
GET /api/items
```

instead of:

```
GET /api/getAllItems
```

REST APIs generally treat `/items` as a resource rather than putting actions into URLs.

Use correct HTTP methods

```
GET    → retrieve  
POST   → create  
PUT    → update  
PATCH → partial update  
DELETE → delete
```

Validate all input

Never blindly trust API input.

```
$request->validate([  
    'name' => 'required|string|max:255'  
]);
```

Protect sensitive routes

```
Route::middleware('auth:sanctum')->group(function () {  
    // protected endpoints  
});
```

Use API Resources

Avoid returning raw models everywhere.

```
return new ItemResource($item);
```

This gives you control over your API output.

Use proper status codes

Don't return:

200

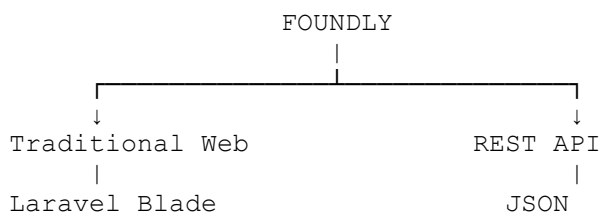
for every situation.

Use:

```
200 → successful retrieval  
201 → created  
401 → unauthenticated  
403 → unauthorized  
404 → not found  
422 → validation error  
500 → server error
```

COMPLETE WEEK 3 + WEEK 4 CONCEPT MAP

The easiest way to understand how everything connects is to think about your **Foundly application**.



Authentication	Sanctum
Middleware	API Routes
Authorization	API Controllers
Roles & Permissions	Validation
Eloquent Relationships	API Resources
File Uploads	Status Codes
Search + Pagination	Postman
Resource Controllers	Testing
Route Model Binding	

MOST IMPORTANT TERMINOLOGY TO REMEMBER FOR VIVA

Term	Professional Definition
Eloquent ORM	Laravel's ORM for interacting with database records through PHP models
Relationship	Association between records in different database tables
Foreign Key	Column referencing a primary key in another table
Authentication	Process of verifying a user's identity
Authorization	Process of determining what an authenticated user is allowed to do
Middleware	Layer that filters HTTP requests before they reach application logic
Role	General category representing a user's responsibilities/access level
Permission	Specific action a user is allowed to perform
Gate	Laravel mechanism for defining authorization rules
Policy	Class containing authorization logic for a specific model
File Upload	Process of transferring a file from client to server
Storage	Laravel's filesystem abstraction for managing files
Pagination	Dividing large datasets into smaller pages
Resource Controller	Controller structured around standard CRUD actions
Route Model Binding	Automatic resolution of route parameters into model instances
API	Interface through which software systems communicate
REST	Architectural style for designing resource-oriented web APIs

Term	Professional Definition
JSON	Lightweight format commonly used for exchanging API data
API Route	HTTP endpoint through which an API can be accessed
Postman	Tool used to develop and test APIs
Sanctum	Laravel package for lightweight API/SPA authentication
API Token	Credential used to authenticate API requests
API Resource	Laravel class that transforms models into structured JSON
HTTP Status Code	Numeric code describing the result of an HTTP request
Validation	Process of ensuring incoming data satisfies defined rules

WEEK 3 VS WEEK 4

The conceptual difference is very important.

Week 3

You are primarily building a **server-rendered web application**:

```

Browser
↓
Laravel Route
↓
Controller
↓
Model / Database
↓
Blade View
↓
HTML
↓
Browser

```

Week 4

You are building an **API**:

```

Client
↓
HTTP Request
↓
API Route
↓
Controller
↓
Model / Database
↓

```

API Resource



JSON



Client

So the biggest transition is:

Week 3 teaches you how to build a structured Laravel web application; Week 4 teaches you how to expose Laravel functionality as a secure, testable REST API.

For **Foundly**, this means your existing lost/found item system can eventually support not only Blade pages but also a React frontend, mobile application, or any other client through the Laravel API.